



3
13
1560 sp

عملي

كلية الهندسة المعلوماتية
السنة الثانية

Binary search tree



د. عملي مشترك

RB Informatics; 24/04/2025
محتوى مجاني غير مخصص للبيع التجاري

خوارزميات ٢

يفترض أننا في المحاضرات السابقة أتقنا التعامل لحد كبير مع الأشجار الثنائية بالعموم في محاضرتنا هذه سنتعرف على نوع خاص من الأشجار الثنائية والذي له أهمية كبيرة في سياق الخوارزميات وله حصة كبيرة مما سنناقش في مقررنا

شجرة البحث الثنائي binary search tree

Definition ↩

نقول عن الشجرة الثنائية أنها binary search tree إذا حققت الشرطين التاليين، لكل عقدة في الشجرة يكون:

- الشجرة التابعة لابنها الأيسر تحوي قيماً أصغر من قيمة العقدة
 - الشجرة التابعة لابنها الأيمن تحوي قيماً أكبر من قيمة العقدة
- وهذا محقق لجميع العقد التي لها أبناء.

لا يوجد فيها عقد ذات قيم مكررة

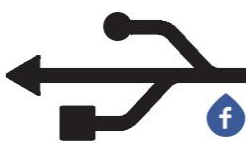
سننتعرف على مفهوم بسيط يسمى التوازن ال balancing ضمن الأشجار، ما معنى

أن تكون الشجرة balanced؟

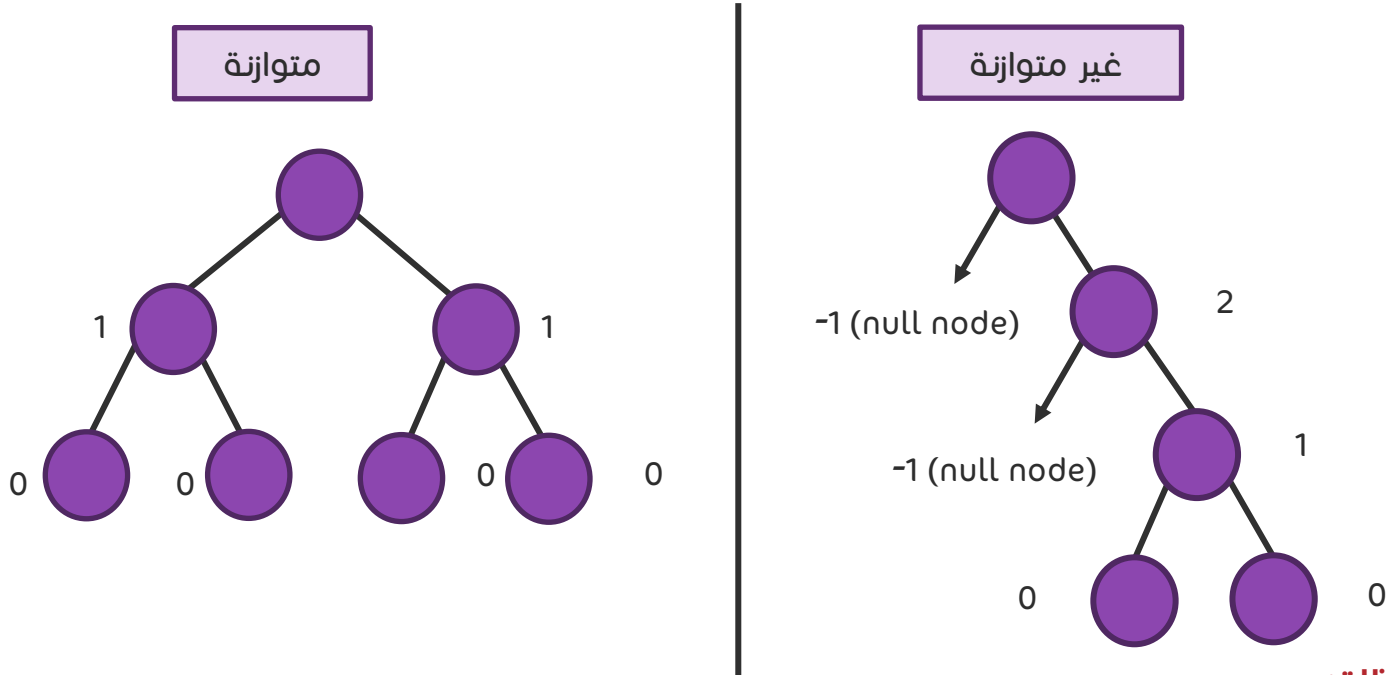
يعني أنه ضمن الشجرة لكل عقدة فيها يكون:

الفرق بين ارتفاع الشجرة التابعة لابنها الأيمن والشجرة التابعة لابنها الأيسر هو قيمة صغيرة ثابتة (0 أو 1) يمكن تعريفها بطريقة أخرى بأن الأوراق في هذه الشجرة تقع جميعها تقريباً في نفس المستوى من الشجرة

وأي شجرة لا تحقق ذلك تكون unbalanced.



مثال: سنضع ارتفاع الشجرة على يمين وعلى يسار كل عقدة حتى يتضح المعنى.



ملاحظات:

1. نعرف ارتفاع العقدة **null** بكونه -1 حتى يتفق مفهوم الارتفاع الذي نفهمه (وكذلك مع التعريف العودي) بحيث أن معنى الارتفاع في الشجرة هو بعد الجذر عن الورقة.
2. لو فرضنا عقدة لا أبناء لها يكون بعدها عن الورقة هو 0 كونها هي بذاتها ستكون جذراً وورقة في آن واحد لنفترض أن هذه العقدة اختفت فالبعد سينقص بمقدار عقدة إضافية (العقدة التي اختفت) بالتالي سيصبح -1
3. الشجر المتوازن وغير المتوازن لا يقتصر على الأشجار الثنائية يمكن تعميمه على أشجار ذوات درجات أعلى

ما سبب تسمية الـ binary search tree بهذا الاسم؟

لأن الشجرة بطبيعتها تدعم عملية البحث بشكل منهجي لحد ما.
 في شجرة ثنائية ذات قيم عشوائية عند البحث عن عنصر سنضطر للمرور على كافة العقد بأسوأ الأحوال بحيث سنستمر باستدعاء التابع على كافة العقد حتى نصل للنتيجة.
 بينما في شجرة البحث الثنائي سيصبح الأمر مبني على: هل القيمة التي أبحث عنها أصغر من قيمة العقدة الحالية؟ أم أكبر من قيمة العقدة الحالية؟ ونميز حالتين:

- في حال كانت القيمة المستهدفة أصغر نذهب إلى الابن الأيسر ونستثني الابن الأيمن
- في حال كانت القيمة أكبر نذهب إلى الابن الأيمن ونستثني الابن الأيسر

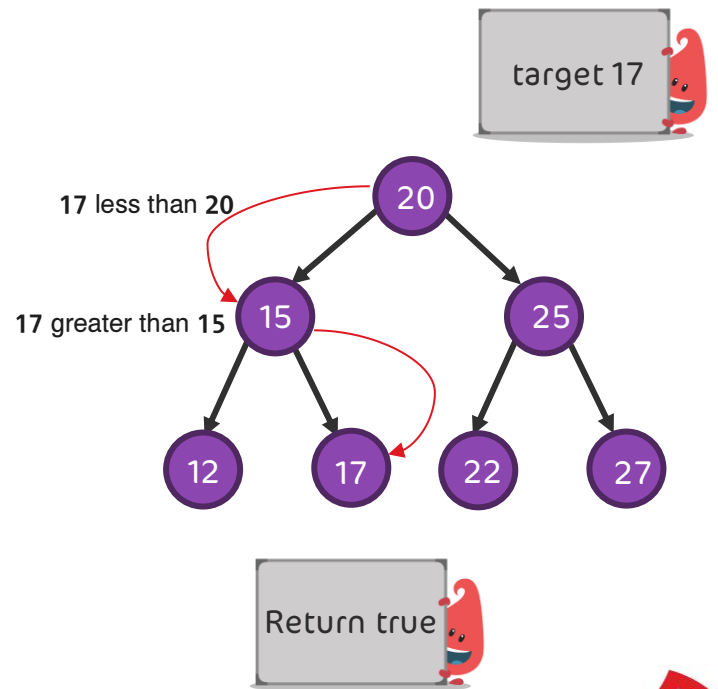
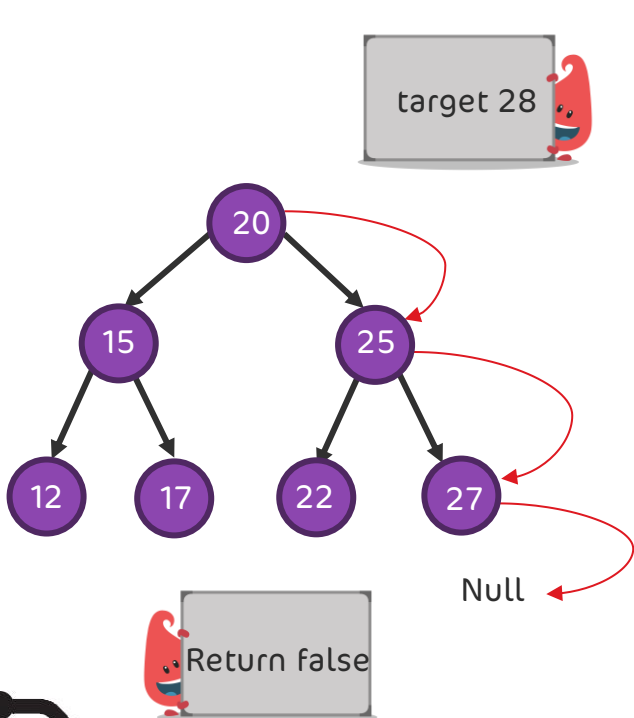
ونطبق ذلك على كل عقدة حتى نصل إلى الناتج.

- البحث بهذه الطريقة يشبه الـ binary search بحيث في كل عملية نتخلص من نصف عينة القيم التي نبحت ضمنها، وهذا يجعل من التعقيد $O(h)$ حيث h هو ارتفاع الشجرة **بحيث سنضطر للمرور على كل مستوى مرة واحدة فحسب.**
- على عكس التعقيد في عملية البحث الأولى في الشجرة الثنائية ذات القيم العشوائية و الذي سيكون $O(n)$ حيث n هو عدد العقد.
- و لكن سبب شرحنا لمفهوم التوازن يرجع إلى نقطة و هي في حال كانت الشجرة الخاصة بالبحث الثنائي متوازنة هنا ستكون: $h = \log(n) + 1$ حيث n تمثل عدد العقد بحيث بكل عملية سنستثني نصف عدد العقد مما يجعل التعقيد لو غارتمني.
- وهنا إضافة $(+1)$ هنا قد تكون للتعامل مع حالات الحافة أو تفضيل تعريف معين للارتفاع.
- بينما لو افترضنا أن الشجرة لا تحقق مفهوم التوازن فعندها يمكن لـ h في أسوأ حالة أن تساوي n (شجرة خطية) بالتالي تصبح عملية البحث مكلفة مجدداً.
- لا يمكن لشجرة البحث الثنائي العادية أن تضمن التوازن و هذه أحد مشكلاتها التي سننتظر كيف يتم حلها في موضوعات المحاضرات اللاحقة.

الآن سنتحدث بشكل عام عن العمليات التي يمكن إجراؤها ضمن هذه الشجرة

وكيف نقوم بـ searching , insertion و deletion.

- العملية الأساسية في شجرة البحث الثنائي هي البحث وشرحنا آلية عملها سابقاً.
- سيعتمد الكود على مقارنة القيمة التي نبحت عنها مع العقدة الحالية، والاتجاه للعقدة اليسرى في حال كانت القيمة التي نبحت عنها أصغر، أو للعقدة اليمنى في حال كانت القيمة التي نبحت عنها أكبر، والتوقف وإرجاع true إذا تساوت قيمة العقدة مع القيمة التي نبحت عنها، وفي حالة وصولنا لـ null نعيد false.



CODE

```

class BinaryTree {
    Node root;
    class Node {
        int value;
        Node left, right;
        Node(int value) {
            this.value = value;
            this.left = this.right = null;
        }
    }
    public boolean search (Node root , int key){
        if(root == null)
            return false;
        if(root.value == key)
            return true;
        else if (key < root.value)
            return search(root.left, key);
        else
            return search(root.right, key);
    }
}

```

1

```

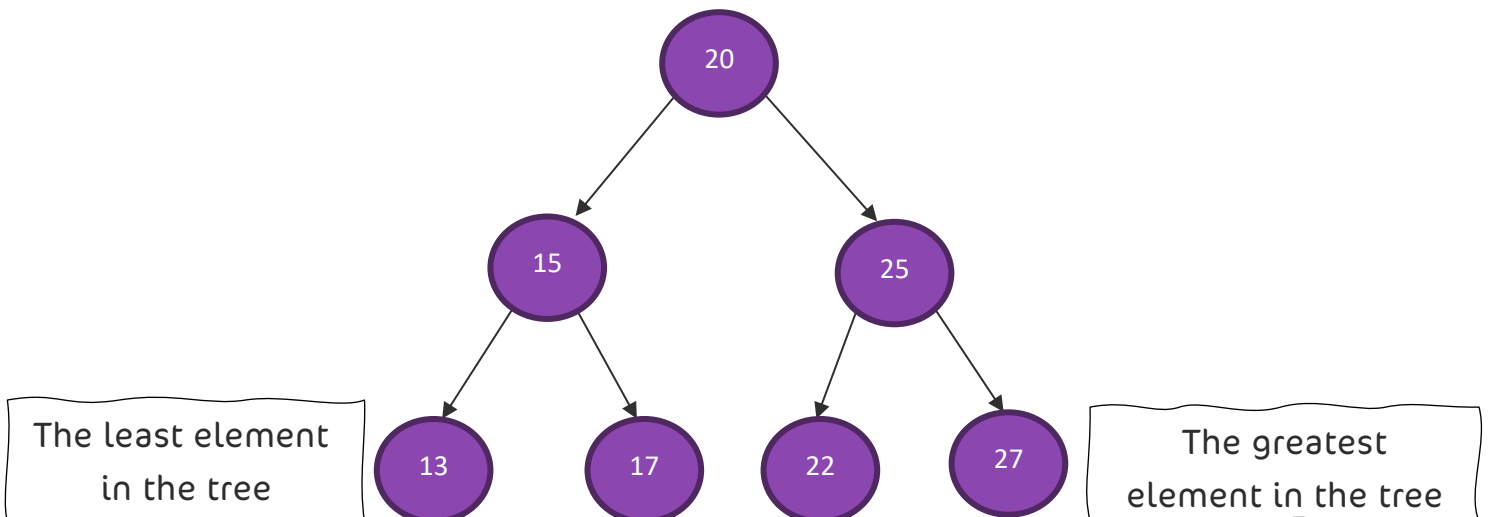
public class Main {
    public static void main(String[] args) {
        BinaryTree bt = new BinaryTree();
        bt.root = bt.new Node(20);
        bt.root.right = bt.new Node(25);
        bt.root.right.left = bt.new Node(22);
        bt.root.right.right = bt.new Node(27);
        bt.root.left = bt.new Node(15);
        bt.root.left.right = bt.new Node(17);
        bt.root.left.left = bt.new Node(13);

        System.out.println(bt.search(bt.root, 27));
    }
}

```

2

- يمكننا العثور على أصغر عنصر في هذه الشجرة من خلال الذهاب للعقدة في أقصى اليسار وهكذا الأمر مع أكبر عنصر في هذه الشجرة يمكننا الحصول عليه من خلال الذهاب للعقدة في أقصى اليمين. يمكن فعل ذلك عودياً بسهولة



CODE

```

class BinaryTree {
    Node root;
    class Node {
        int value;
        Node left, right;
        Node(int value) {
            this.value = value;
            this.left = this.right = null;
        }
    }
    public int minElement(Node root){
        if(root.left==null)
            return root.value;
        return minElement(root.left);
    }
    public int maxElement(Node root){
        if(root.right==null)
            return root.value;
        return maxElement(root.right);
    }
}

```

```

public class Main {
    public static void main(String[] args) {
        BinaryTree bt = new BinaryTree();
        bt.root = bt.new Node(20);
        bt.root.right = bt.new Node(25);
        bt.root.right.left = bt.new Node(22);
        bt.root.right.right = bt.new Node(27);
        bt.root.left = bt.new Node(15);
        bt.root.left.right = bt.new Node(17);
        bt.root.left.left = bt.new Node(13);
        System.out.println(bt.minElement(bt.root));
        System.out.println(bt.maxElement(bt.root));
    }
}

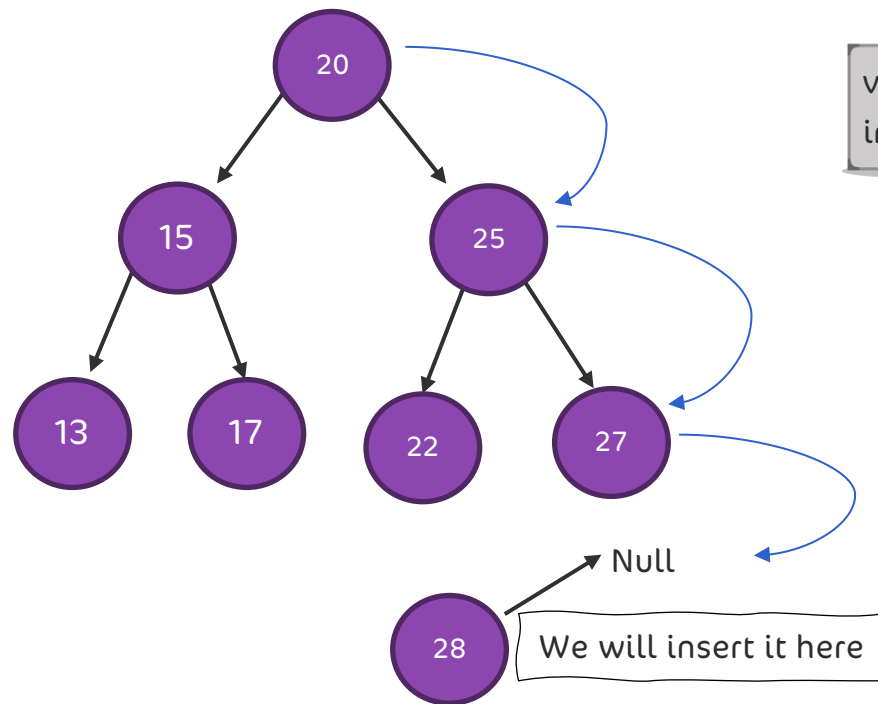
```

حتى نضيف عنصر على شجرة البحث الثنائي، هذا العنصر المضاف يجب ألا يخرق قواعد الشجرة التي ذكرناها:

1- ألا يكون العنصر موجوداً مسبقاً ضمن الشجرة.

2- أن يحافظ على فكرة أن الشجرة اليمين فيها قيم أكبر من قيم العقدة الحالية والشجرة اليسار فيها قيم أصغر من العقدة الحالية.

وللقيام بذلك سنتبع نفس منطق البحث حتى نجد عقدة null في هذه الحالة سنجعل هذه العقدة تساوي القيمة الجديدة المسندة (يعني عند كل عقدة نرى هل القيمة التي نود إسنادها أكبر أم أصغر من قيمة العقدة الحالية ونذهب لليمين إن كانت أكبر ولليسار إن كانت أصغر في نهاية المطاف سنصل ل null نعدل قيمته حتى يحمل قيمة العقدة الجديدة وبالتالي ستتحقق شروط شجرة البحث الثنائية).



CODE

```

class BinaryTree {
    Node root;
    class Node {
        int value;
        Node left, right;
        Node(int value) {
            this.value = value;
            this.left = this.right = null;
        }
    }
    public Node insert (Node root,int val){
        if (root == null)
            return new Node(val);
        if (root.value>val)
            root.left = insert(root.left,val);
        else
            root.right =
insert(root.right,val);
        return root;
    }
}
    
```

```

public class Main {
    public static void main(String[] args) {
        BinaryTree bt = new BinaryTree();
        bt.root = bt.new Node(20);
        bt.root.right = bt.new Node(25);
        bt.root.right.left = bt.new Node(22);
        bt.root.right.right = bt.new Node(27);
        bt.root.left= bt.new Node(15);
        bt.root.left.right = bt.new Node(17);
        bt.root.left.left = bt.new Node(13);
        bt.insert(bt.root,28);
        System.out.println(bt.root.right.right.right.value);
    }
}
    
```

- في حال أردنا حذف عقدة من شجرة بحث ثنائية للتخلص من هذا العنصر يجب ألا يؤثر على بنية شجرة البحث و يجب أن تبقى محافظة على الشروط الخاصة بشجرة البحث الثنائية كما في عملية الإضافة، يوجد لدينا ثلاثة حالات أثناء الحذف سنعالج كل واحدة على حدى بحيث تستمر الشجرة بصورة صحيحة:

الحالة الأولى:

- العقدة التي نود حذفها ورقة: عند إذ فقط نحذفها دون أي إجراء آخر.

الحالة الثانية:

- العقدة التي نود حذفها لها ابن وحيد: عندئذ نجعل ابنها يحل مكانها فحسب.

الحالة الثالثة:

- العقدة التي نود حذفها لها ابنان: نذهب إلى أكبر عقدة في شجرة الابن الأيسر ونضعه مكان العقدة بحيث هذا العنصر تحديداً سيرا على شروط الشجرة 100% ومن الممكن كذلك أن نذهب إلى أصغر عقدة في شجرة الابن الأيمن ونضعه مكان العقدة وينطبق عليه الكلام السابق، لما ذلك؟
- لأن أكبر عنصر في شجرة الابن الأيسر هو أكبر من جميع العناصر فيها (left subtree) و لكنه حتماً أصغر من الجذر (لأنه ضمن الشجرة التابعة للابن الأيسر للعقدة وفقاً لتعريف شجرة البحث الثنائي يجب أن تحوي دوماً قيماً أصغر من العقدة التي نقف عندها)، و كذلك الأمر بالنسبة لأصغر عنصر في شجرة الابن الأيمن ؛ لأن أصغر عنصر في شجرة الابن الأيمن هو أصغر من جميع العناصر فيها و لكنه حتماً أكبر من الجذر (لأنه ضمن الشجرة التابعة للابن الأيمن للعقدة وفقاً لتعريف شجرة البحث الثنائي يجب أن تحوي دوماً قيماً أكبر من العقدة التي نقف عندها) و نحن نود وضع هذا العنصر مكان الجذر بعد عملية الحذف.

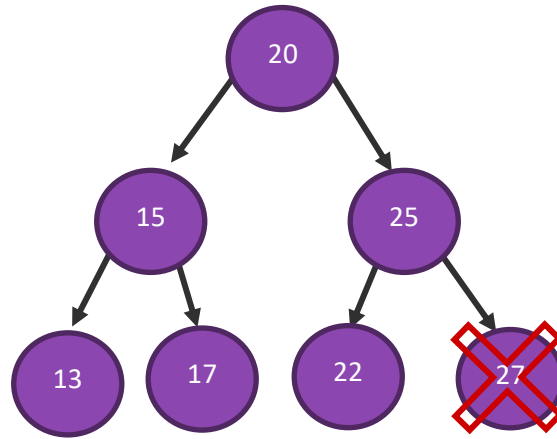
الحالة الأولى فقط نسند العقدة ل null

الحالة الثانية فقط نسند العقدة للابن الموجود

الحالة الثالثة نبحت عن أحد العنصرين المميزين الذين ذكرناهما ثم نأخذ قيمته ونضعها في العقدة التي

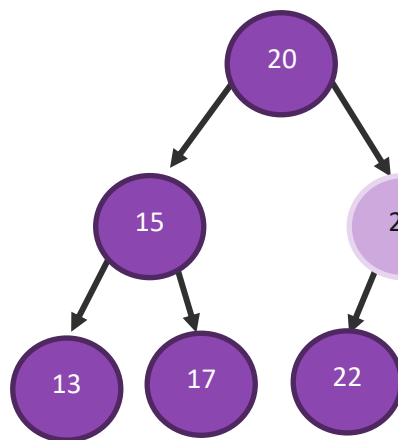
طلب حذفها، ثم نذهب إلى مكان هذا العنصر المميز ونحذفه

- (بالكود القادم سوف نذهب لأصغر عنصر في الشجرة اليمنى نأخذ قيمته ونضعها في العقدة الحالية "المطلوب حذفها" ثم نحذف هذا العنصر الأصغر من الشجرة اليمنى " وهو محقق للحالة الأولى")



value to delete: 27

Assign this to null



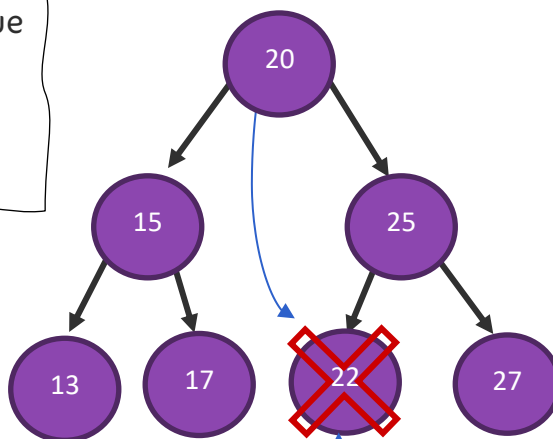
This would be 22

value to delete: 25

Assign the child to the parent

This would be null

Assign this to minimum value of the right tree
Then delete the minimum right node



value to delete: 20

This would be null

CODE

```

class BinaryTree {
    Node root;
    class Node {
        int value;
        Node left, right;
        Node(int value) {
            this.value = value;
            this.left = this.right = null;
        }
    }
}

```

```

public class Main {
    public static void main(String[] args) {
        BinaryTree bt = new BinaryTree();
        bt.root = bt.new Node(20);
        bt.root.right = bt.new Node(25);
        bt.root.right.left = bt.new Node(22);
        bt.root.right.left.left = bt.new Node(21);
        bt.root.right.right = bt.new Node(27);
        bt.root.left = bt.new Node(15);
        bt.root.left.right = bt.new Node(17);
        bt.root.left.left = bt.new Node(13);
        System.out.println(bt.root.value);
        bt.delete(bt.root, 20);
        System.out.println(bt.root.value);
    }
}

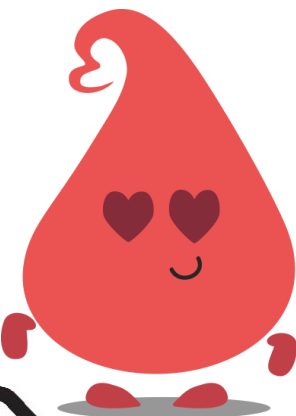
```

```

public Node minElement(Node root){
    if(root.left!=null)
        return minElement(root.left);
    else
        return root;
}

public Node delete (Node root,int val){
    // if we don't find the val
    if (root == null)
        return null;
    if(val < root.value)
        root.left = delete(root.left,val);
    else if(val > root.value)
        root.right = delete(root.right,val);
    else {
        // if no children
        if(root.left==null && root.right==null)
            return null;
        //if only one child
        else if (root.left==null)
            return root.right;
        else if (root.right ==null)
            return root.left;
        //if two children
        else {
            Node node =minElement(root.right);
            root.value = node.value;
            root.right=delete(root.right,node.value);
        }
    }
    return root;
}
}

```



EXERCISE 1

ليكن لدينا شجرة ثنائية اكتب تابع يتأكد من كون هذه الشجرة شجرة بحث ثنائي؟

فكرة الحل:

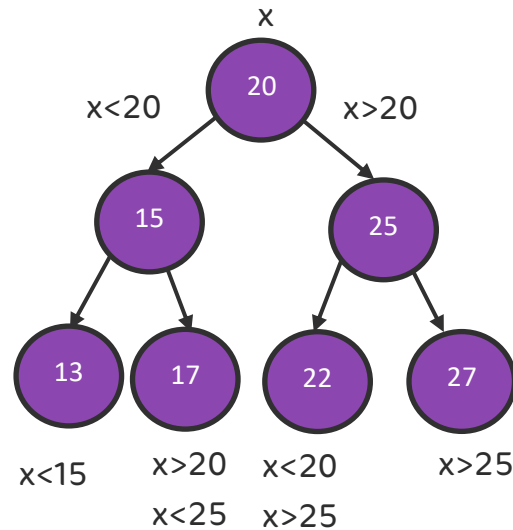
سنعتمد على مجال القيم المسموحة لكل عقدة من العقد، لنفترض لدينا جذر وله اثنان مجال القيم المسموحة للجذر هو: أي قيمة بين $]-\infty, +\infty[$ ، بفرض الجذر a .
(لا يوجد حدود للقيمة التي تملكها أول عقدة في أي شجرة ابتداءً حتى تكون شجرة بحث ثنائي).
بعد تعيين قيمة الجذر (a) لكل عقدة من عقد الأبناء سنضيف المجال المسموح من القيم بحيث تبقى الشجرة شجرة بحث ثنائي ، فيجب الآن على الابن الأيمن أن تكون قيمته هي قيمة محصورة بين قيمة الجذر و اللانهاية $]a, +\infty[$ (أي قيمة أكبر تماماً من قيمة الجذر)
و يجب كذلك على الابن الأيسر أن تكون قيمته هي قيمة محصورة بين السالب لا نهاية وقيمة الجذر $]-\infty, a[$ (أي قيمة أصغر تماماً من قيمة الجذر)
و هذا النمط يعمم فنجد أننا بالنسبة لكل شجرة جزئية من الشجرة الكلية نطبق نفس الفكرة بحيث يكون لدينا مجال من القيم لو حملت العقدة أيأ منها تبقى الشجرة تحقق شروط شجرة البحث الثنائي
و نختبر قيمة العقدة هل هي موجودة ضمن هذا المجال و إن وجد ضمنه فنعيد حصر المجال بناء على قيمة هذه العقدة الجديدة

ملاحظة:

لا يمكن لقيمة العقدة التي نختبرها أن تساوي قيم أطراف المجال وذلك لكيلا تخرق الشرط الخاص "بتكرار القيم ضمن الشجرة" وإن لم تكن موجودة فهذه الشجرة ليست شجرة بحث ثنائي.

كيف يتم تعديل المجال بناء على قيمة العقدة؟

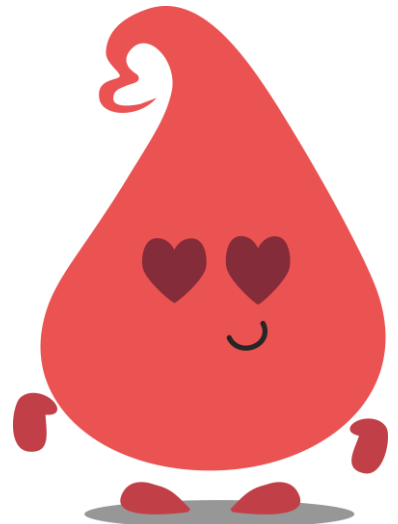
عند كل عقدة تحقق انتماءها للمجال الذي نفحصه يجب على ابنها الأيسر أن يكون أصغر منها حتماً (فتصبح أقصى قيمة للمجال و التي يمكن أن يصل لها الابن الأيسر هو قيمة العقدة نفسها "مطروحاً منها واحد لكي لا تصبح قيمة موجودة مسبقاً " و الحد الأدنى يبقى على ما كان عليه في الحالة السابقة).
و يجب كذلك على الابن الأيمن أن يكون أكبر منها حتماً (فتصبح أدنى قيمة ضمن المجال و التي يمكن أن يصلها الابن الأيمن هو قيمة العقدة نفسها "مضافاً لها واحد لكيلا تصبح قيمة موجودة مسبقاً " و الحد الأعلى يبقى على ماكن عليه في الحالة السابقة).



CODE

```

class BinaryTree {
    Node root;
    class Node {
        int value;
        Node left, right;
        Node(int value) {
            this.value = value;
            this.left = this.right = null; }
    public boolean isBST (Node root,int min , int max){
        if (root==null)
            return true;
        if(root.value<=min || root.value>=max)
            return false;
        return isBST(root.left,min,root.value)&&isBST(root.right,root.value,max);}}
    public class Main {
        public static void main(String[] args) {
            BinaryTree bt = new BinaryTree();
            bt.root = bt.new Node(20);
            bt.root.right = bt.new Node(25);
            bt.root.right.left = bt.new Node(22);
            bt.root.right.left.left = bt.new Node(21);
            bt.root.right.right = bt.new Node(27);
            bt.root.left= bt.new Node(15);
            bt.root.left.right = bt.new Node(17);
            bt.root.left.left = bt.new Node(13);
            System.out.println(bt.isBST(bt.root,Integer.MIN_VALUE,Integer.MAX_VALUE)); }
    }
  
```



EXERCISE 2

أثبت أن لشجرة البحث الثنائية نهاية مية؟

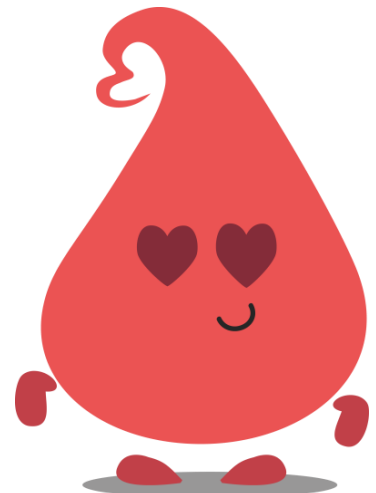
تكون شجرة البحث الثنائي تمتلك نهاية مية إن كان في الشجرة عقدة لا يمكن لنا إضافة أي عقدة جديدة لها دون خرق أحد شروط شجرة البحث الثنائية (للوصول الى هكذا أمر لنفترض أننا نخزن فقط قيمة موجبة ضمن الشجرة وبالتالي سيكون المجال البدائي للجذر هو $[1, +\infty[$

فكرة الحل:

اعتماداً على التمرين السابق: الحالة الوحيدة التي لا يمكن لنا فيها إضافة قيمة هي عندما يصبح مجال القيم المسموح للعقدة التالية نهايته تنطبق على بدايته ($\max = \min$) بحيث لن يعود من الممكن إيجاد أي قيمة ضمن هذا المجال يمكن اختيارها بحيث تبقى الشجرة تحقق شروط شجرة البحث الثنائية.

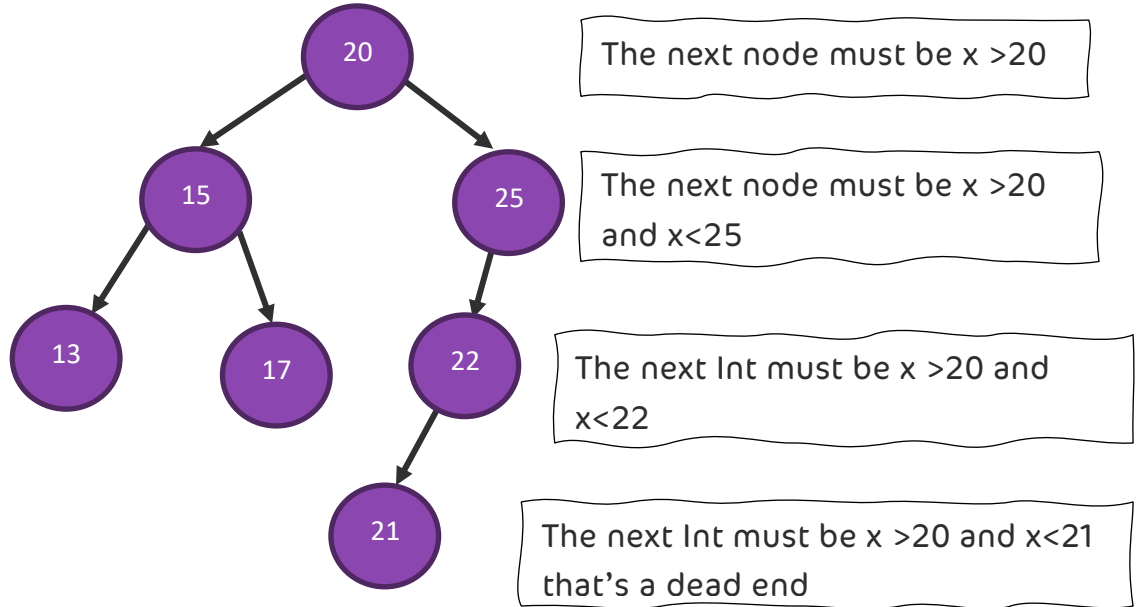
```
class BinaryTree {
    Node root;
    class Node {
        int value;
        Node left, right;
        Node(int value) {
            this.value = value;
            this.left = this.right = null;
        }
    }
    public boolean hasDeadEnd (Node root, int min , int max){
        if (root == null)
            return false;
        if (min == max)
            return true;
        return hasDeadEnd(root.left,min,root.value-1)||hasDeadEnd(root.right,root.value+1,max);
    }
}

public class Main {
    public static void main(String[] args) {
        BinaryTree bt = new BinaryTree();
        bt.root = bt.new Node(20);
        bt.root.right = bt.new Node(25);
        bt.root.right.left = bt.new Node(22);
        bt.root.right.left.left = bt.new Node(21);
        bt.root.right.right = bt.new Node(27);
        bt.root.left = bt.new Node(15);
        bt.root.left.right = bt.new Node(17);
        bt.root.left.left = bt.new Node(13);
        System.out.println(bt.hasDeadEnd(bt.root,1,Integer.MAX_VALUE));
    }
}
```



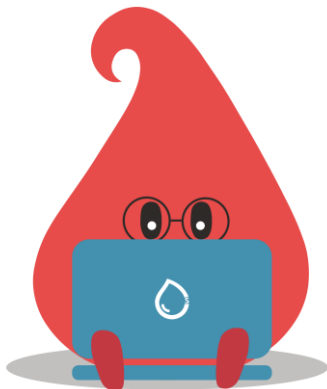
أي انه العقدة تصبح نهاية ميتة عندما لا يوجد أي عدد صحيح موجب بين $\min$ و $\max$ يمكن إضافته كابن للعقدة، والشرط $\min == \max$ يعبر بدقة عن هذه الحالة.

■ شجرة فيها dead end المجال الخاص بالجذر $[1, +\infty]$



وظيفة سيتم حلها بملحق تمارين المحاضرة

- إثبات أن عقدة ما هي سلف لعقدة أخرى (الدخل: قيمتا عقدتين والجذر) (الخرج: true/false)
- إيجاد السلف المشترك لعقدتين (الدخل: قيمتا عقدتين والجذر) (الخرج: قيمة عقدة السلف)



سؤال للتفكر

ما هو نوع الشجرة التي سنحصل عليها من ناحية التوازن، عند عمل شجرة جديدة والبدء بإضافة قيم مرتبة تصاعدياً أو تنازلياً إلى شجرة البحث الثنائي، وما تعقيد عمليات البحث والإضافة والحذف في هذه الشجرة الناتجة؟

THE END